Mise en place d'un cryptosystème d'ElGamal sur les courbes elliptiques et étude de sa sécurité.

Hugo Communal

TIPE
May 13, 2026

Sommaire

1. Construction d'un groupe sur les courbes elliptiques
   - Les courbes elliptiques
   - Plan projectif et point à l'infini
   - Construction de la loi de groupe
   - Construction d'un groupe cyclique
2. Cryptosystème d'ElGamal
   - ElGamal
   - Logarithme discret
3. Attaque du chiffrement
   - Attaque par force brute
   - Algorithme du rho de Pollard
   - Pollard's kangaroo
4. Comparaison des algorithmes
   - Analyse de la sécurité des courbes elliptiques
   - $\mathbb{Z}/p\mathbb{Z}$ vs courbes elliptiques
5. Annexes

Les courbes elliptiques

Une courbe elliptiques est l'ensemble des points de coordonnées (x, y) vérifiant l'équation :

$$ax^3 + bx^2y + cxy^2 + dy^3 + ex^2 + fxy + gy^2 + hx + iy + j = 0$$

où $a, b, c, d, e, f, g, h, i, j \in \mathbb{R}$ et $x, y \in \mathbb{K}$ où $\mathbb{K}$ est un corps.

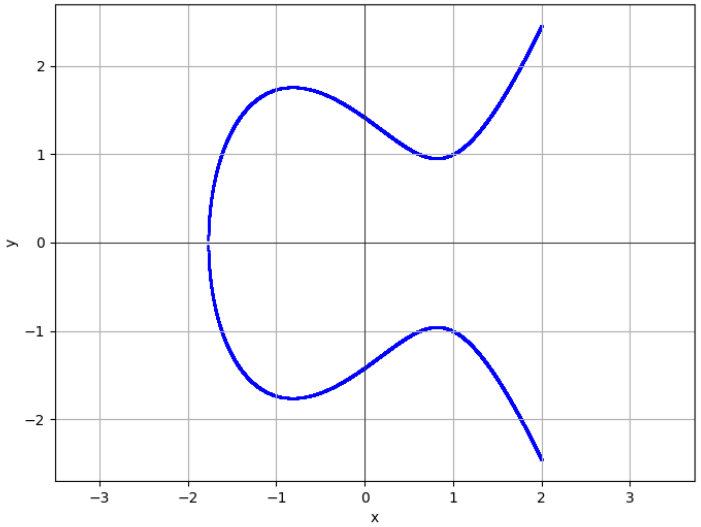


Figure: Courbe elliptique $y^2 = x^3 - 2x + 2$.

Plan projectif et point à l'infini

Definition

Le plan projectif $\mathbb{P}^2$ est l'ensemble des triplets de coordonnées homogènes (x, y, z) où $(x, y, z) \in \mathbb{K}^3 \setminus \{(0, 0, 0)\}$.

Definition

Dans cette notation, les points admettant des coordonnées de la forme $(x, y, 0)$ forment la droite à l'infini. Nous allons associer à la droite à l'infini un point particulier, appelé point à l'infini.

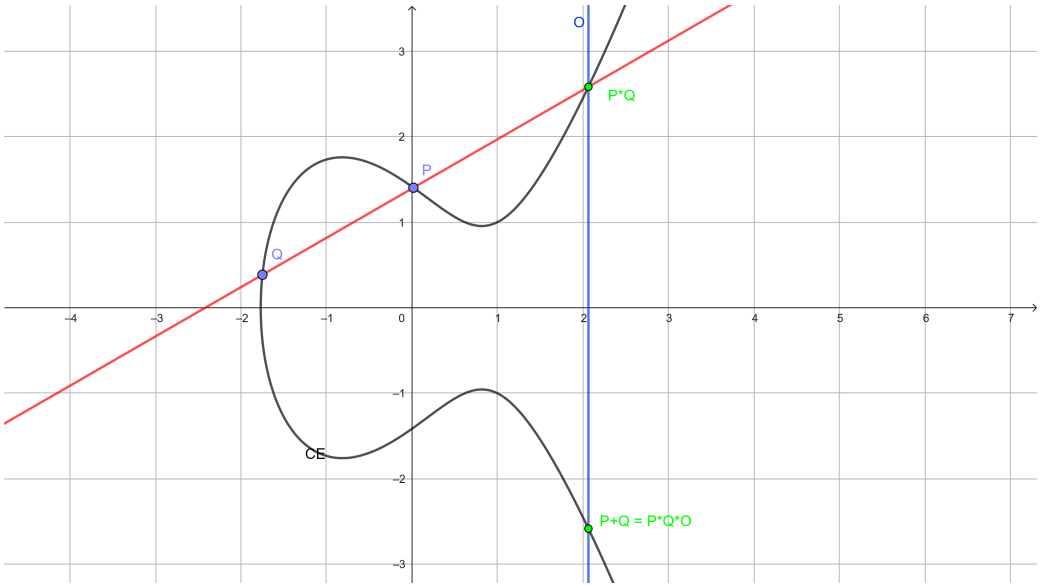
Forme de Weierstrass : Les équation des courbes elliptiques définies sur $\mathbb{R}$ (ou $\mathbb{Q}$) peuvent s'écrire sous la forme suivante

$$\boxed{y^2 = x^3 + ax + b}$$

où $a, b \in \mathbb{R}$ (ou $\mathbb{Q}$) telles que $(4a^3 + 27b^2) \neq 0$ pour que la courbe soit non singulière.

Construction de la loi de groupe

Nous allons définir une loi de groupe additive géométriquement sur les points d'une courbe elliptique dans le plan projectif $(\mathbb{P}^2(\mathbb{E}), +)$.



Cette loi de group est définie de la mainière suivante :

- $P * Q = R$ où R est le troisième point d'intersection de la droit (PQ) avec la courbe elliptique.
- $P + Q = P * Q * O$ où O est le point à l'infini.

Formalisation des propriétés géométriques

Pour $P = (x_1, y_1)$ et $Q = (x_2, y_2)$, en étudiant des égalités de polynômes, on obtient les formules suivantes :

$$\boxed{\begin{array}{rcl} P * Q & = & (x_3, y_3) \\ P * O & = & (x_1, -y_1) \\ P + Q & = & (x_3, -y_3) \\ -P & = & (x_1, -y_1) \end{array}}$$

avec : $\begin{cases} x_3 = \lambda^2 - x_1 - x_2 \\ y_3 = \lambda(x_1 - x_3) - y_1 \end{cases}$ où $\lambda = \frac{y_1 - y_2}{x_1 - x_2}$ et $\nu = y_1 - \lambda x_1$

Ce qui forme bien une loi de groupe associative, commutative avec pour neutre O et qui pour tout point P admet un inverse $-P$.

Construiction d'un groupe cyclique

Definition

Un groupe cyclique est un groupe à la fois monogène et fini. Il existe un élément g du groupe tel que tous les éléments du groupe soient un multipel de g.

Pour tout groupe cyclique $\mathbb{G}$ il existe $n \in \mathbb{N}$ tel que $\mathbb{G} \sim \mathbb{Z}/n\mathbb{Z}$.

Application aux courbes elliptiques

Pour chaque couple d'entiers (x, y) vérifiant l'équation de la courbe elliptique, on applique un modulo à x et à y. L'équation de la courbe devient donc :

$$y^2 = x^3 + ax + b \mod n$$

Exemple de courbe elliptique

L'exemple le plus courant d'utilisation des courbes elliptique est secp256k1, notamement utilisé dans le bitcoin pour vérifier que les fonds sont uniquement utilisés par leur propriétaire.

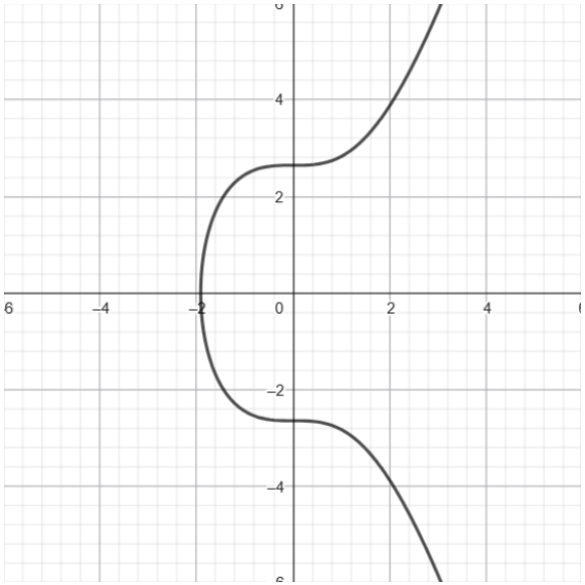


Figure: Courbe elliptique $y^2 = x^3 + 7$.

avec $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ et $p = 2^{256} - 2^{32} - 977$ qui est premier.

ElGamal

Problème : Bob veut envoyer un message à Alice de manière sécurisée sans avoir au préalable échangé une clé de déchiffrement secrète.

Pour résoudre ce problème, Taher Elgamal a proposé en 1984 un protocole qui contrairement à RSA (1977), est libre de droit. De plus, il offre des clés de chiffrement plus courtes que RSA pour un même niveau de sécurité.

Cryptosystème d'ElGamal

Alice choisit une clé secrète $s \in \mathbb{N}$.
Alice partage à Bob (en clair) :

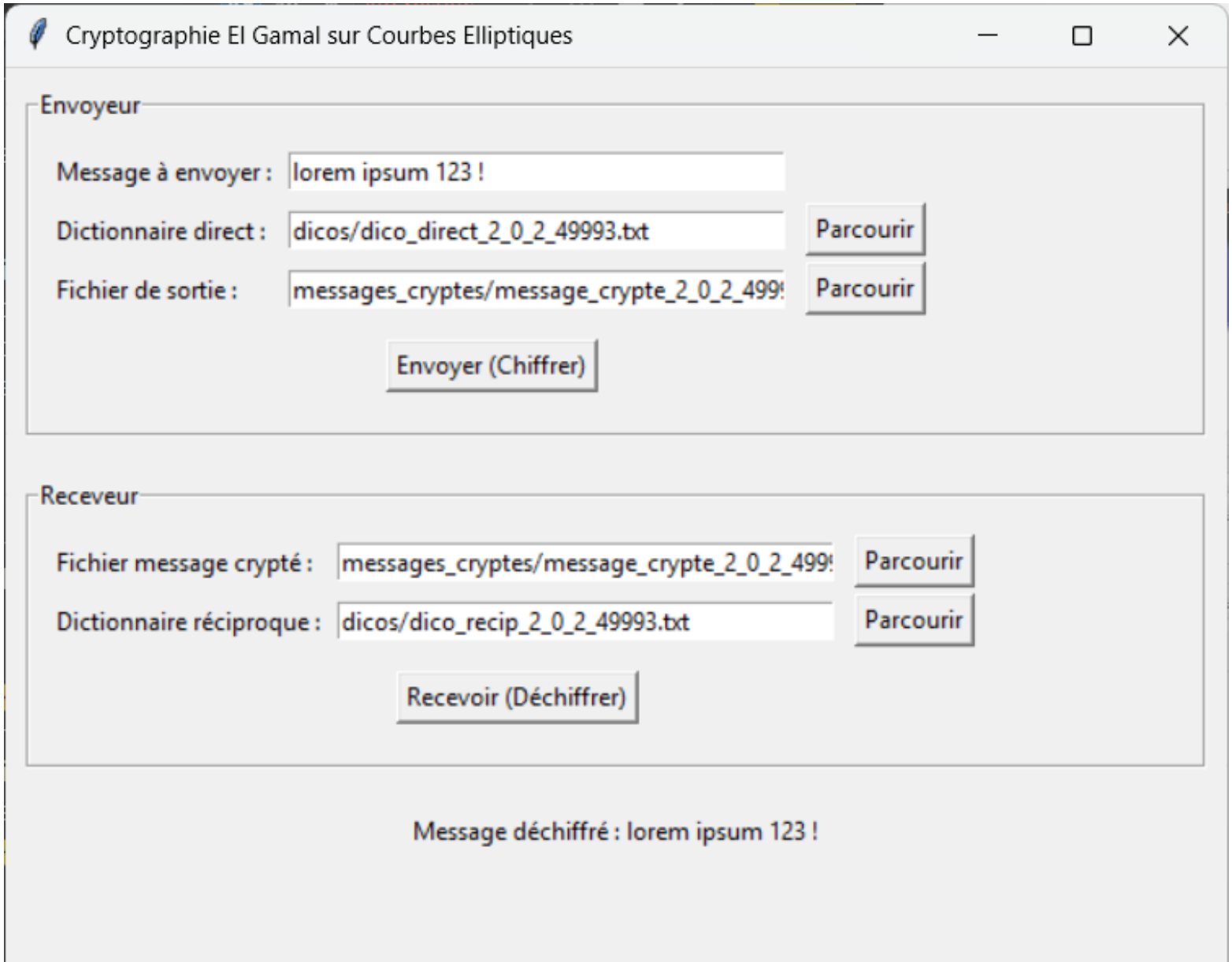
- la donnée d'un groupe $\mathbb{G}$
- un générateur $G \in \mathbb{G}$ du groupe
- la clé publique $P = s \cdot G$

Bob choisit un entier aléatoire $k \in \mathbb{N}$ et un message sous la forme d'un point $M \in \mathbb{G}$.
Bob partage à Alice :

- $M_1 = k \cdot G$
- $M_2 = M + k \cdot P$

<u>Déchiffrement :</u>
Alice calcule $M = M_2 - s \cdot M_1 = M + s \cdot k \cdot G - s \cdot k \cdot G$.

Messagerie


Cryptographie El Gamal sur Courbes Elliptiques
Envoyeur
Message à envoyer :
lorem ipsum 123 !
Dictionnaire direct :
dicos/dico_direct_2_0_2_49993.txt
Parcourir
Fichier de sortie :
messages_cryptes/message_crypte_2_0_2_499
Parcourir
Envoyer (Chiffrer)
Receveur
Fichier message crypté :
messages_cryptes/message_crypte_2_0_2_499
Parcourir
Dictionnaire réciproque :
dicos/dico_recip_2_0_2_49993.txt
Parcourir
Recevoir (Déchiffrer)
Message déchiffré : lorem ipsum 123 !

Logarithme discret

Logarithme discret

fonction réciproque de l'exponentiation discrète :

$$k \mapsto b^k \pmod{n}$$

Soient $a, b \in \mathbb{Z}/n\mathbb{Z}$, $k \in \mathbb{N}$ tels que k est le plus petit entier naturel vérifiant

$$a = b^k,$$

k est donc appelé le **logarithme discret** en base b.

Exemple : Dans $\mathbb{Z}/7\mathbb{Z}$:

$$\log_3(6) = 3 \quad \text{car} \quad 3^3 \bmod 7 = 27 \bmod 7 = 6$$

Attaque par force brute

Objectif : trouver s tel que $P = s \cdot G$

Idée

On additionne successivement le point G jusqu'à obtenir P.

```
s ← 1
X ← G
while X ≠ P do
    if s ≥ o then
        retourner None
    end if
    X ← X + G
    s ← s + 1
end while
retourner s
```

Algorithme du rho de Pollard

Soit une partition $\mathbb{G} = S_0 \cup S_1 \cup S_2$.
On définit une fonciton pseudo aléatoire $f : \mathbb{G} \to \mathbb{G}$ par

$$f(x) = \begin{cases} P + x & x \in S_0 \\ 2x & x \in S_1 \\ G + x & x \in S_2 \end{cases}$$

et les fonctions de $g : \mathbb{G} \times \mathbb{Z} \to \mathbb{Z}$ et $h : \mathbb{G} \times \mathbb{Z} \to \mathbb{Z}$:

$$g(x, a) = \begin{cases} a & x \in S_0 \\ 2a \pmod{n} & x \in S_1 \\ a + 1 \pmod{n} & x \in S_2 \end{cases} \quad h(x, b) = \begin{cases} b + 1 \pmod{n} & x \in S_0 \\ 2b \pmod{n} & x \in S_1 \\ b & x \in S_2 \end{cases}$$

Algorithme du rho de Pollard

Entrées : G générateur de $\mathbb{G}$, $P \in \mathbb{G}$
Sortie : s tel que $P = s \cdot G$, ou échec

```
n ← ordre(G)
Tirer a, b ← {0, ..., n − 1}
x ← aG + bP,   X ← x,   A ← a,   B ← b
while True do
    (x, a, b) ← (f(x), g(x, a), h(x, b))
    (X, A, B) ← (f(X), g(X, A), h(X, B))
    (X, A, B) ← (f(X), g(X, A), h(X, B))
    if x = X then                                   ▷ collision ?
        r ← (b − B) mod n
        if r = 0 then
            return échec
        end if
        return r^{−1} · (A − a) mod n               ▷ calcul de s
    end if
end while
```

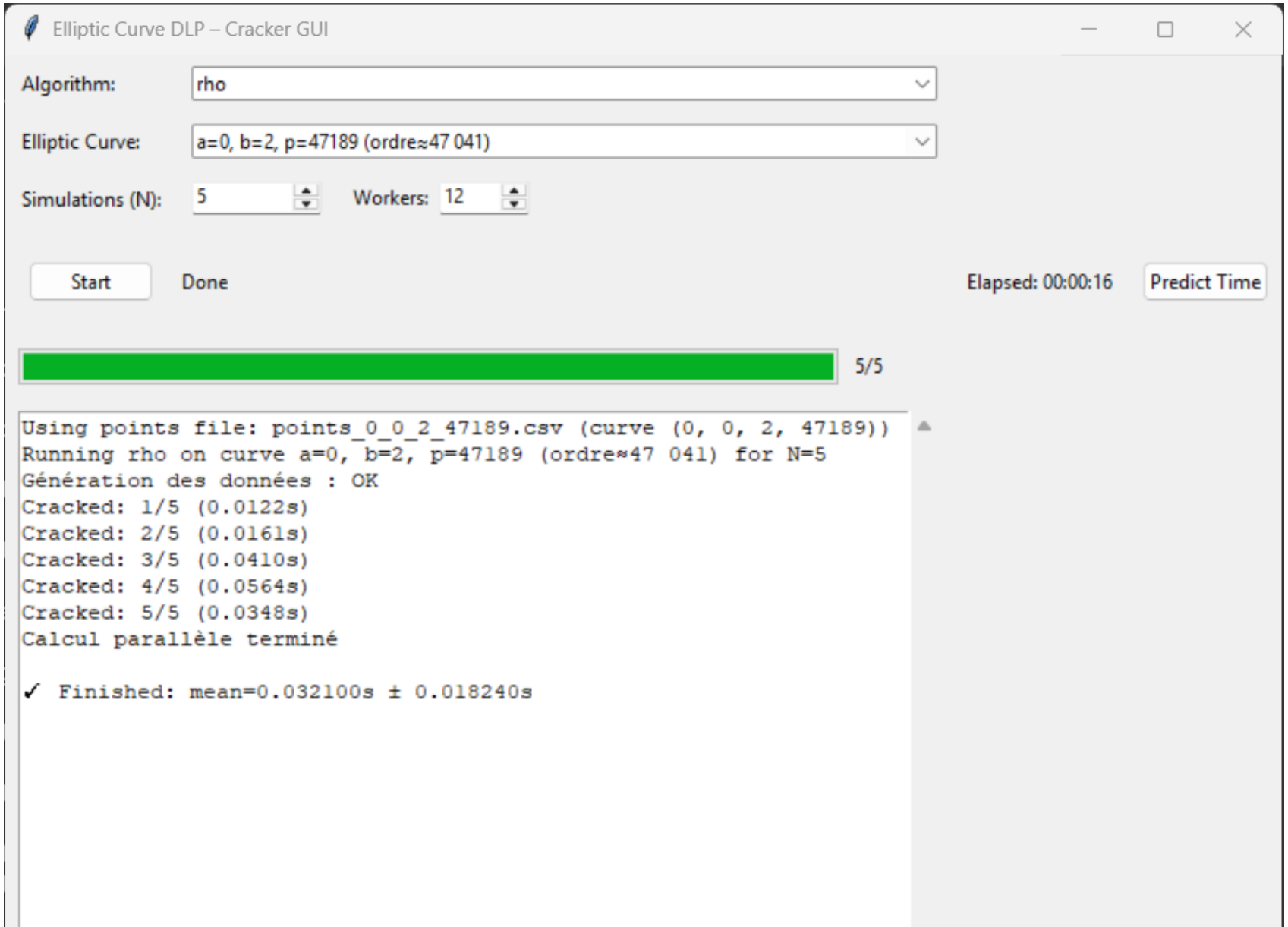
Pollard's kangaroo

<u>intervalle de recherche</u> : on sait que $s \in [a, b]$, complexité en $O\left(\exp\left(\frac{b-a}{2}\right)\right)$.

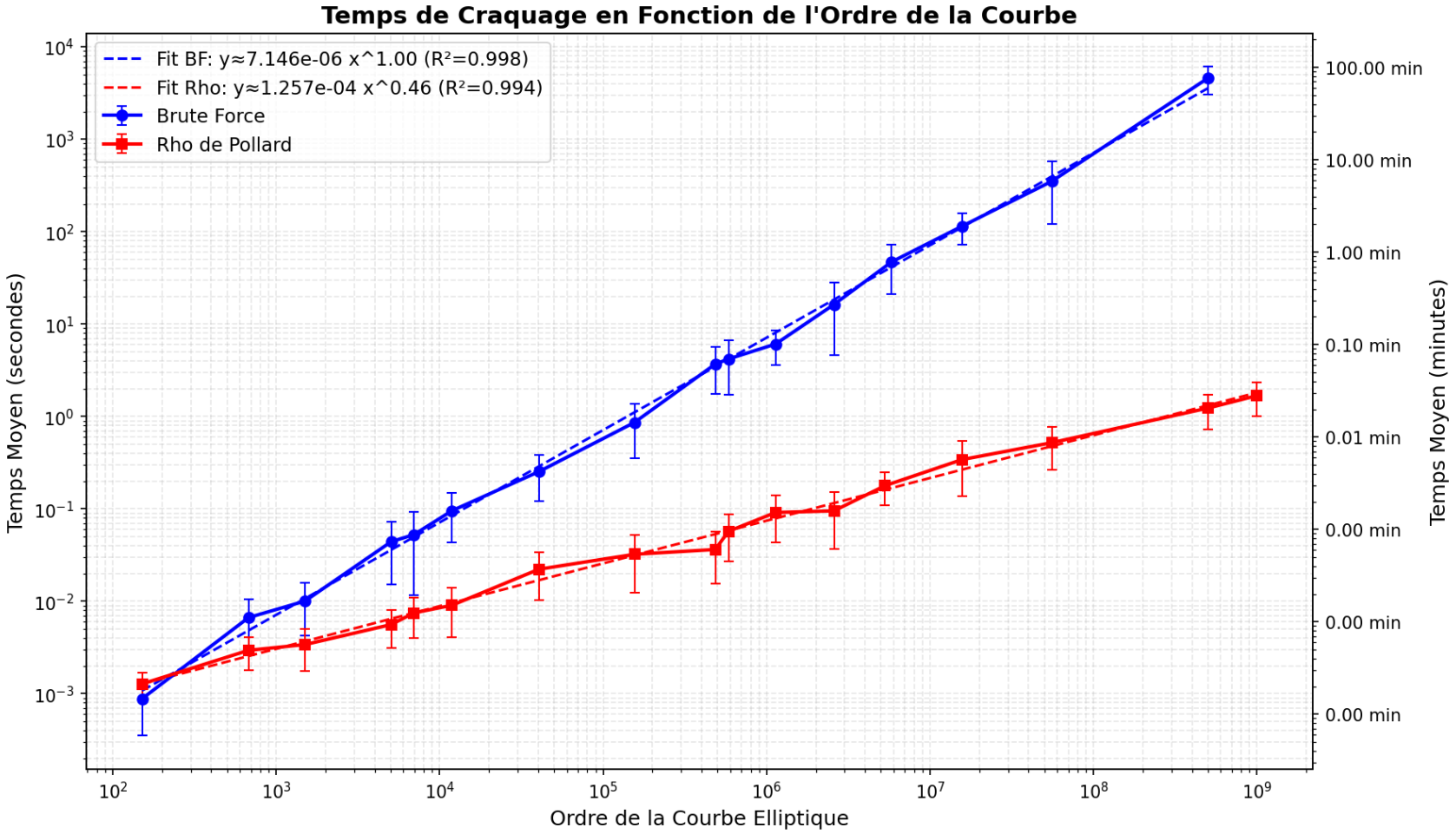
$N \leftarrow \lfloor \sqrt{b-a} \rfloor + 1$
$f(x) \leftarrow (x.x + x.y) \bmod N + 1$ ▷ fonction de saut
$x \leftarrow bG, \quad d_x \leftarrow 0$ ▷ kangourou apprivoisé
for $i = 0$ **à** $N - 1$ **do**
    $d_x \leftarrow d_x + f(x)$
    $x \leftarrow x + f(x) \cdot G$
end for
$y \leftarrow P, \quad d_y \leftarrow 0$ ▷ kangourou sauvage
for $i = 0$ **à** $4N - 1$ **do**
    if $y = x$ **then** ▷ les kangourous se rencontrent
        return $(b + d_x - d_y) \bmod n$
    end if
    $d_y \leftarrow d_y + f(y)$
    $y \leftarrow y + f(y) \cdot G$
    if $d_y > b - a + d_x$ **then**
        return `échec`
    end if
end for

- Pourquoi le logarithme discret est-il considéré comme un problème difficile ?
- Pourquoi utilise t-on les courbes elliptiques ?

Protocole


Elliptic Curve DLP – Cracker GUI
Algorithm:
rho
Elliptic Curve:
a=0, b=2, p=47189 (ordre≈47 041)
Simulations (N):
5
Workers:
12
Start
Done
Elapsed: 00:00:16
Predict Time
5/5
Using points file: points_0_0_2_47189.csv (curve (0, 0, 2, 47189))
Running rho on curve a=0, b=2, p=47189 (ordre≈47 041) for N=5
Génération des données : OK
Cracked: 1/5 (0.0122s)
Cracked: 2/5 (0.0161s)
Cracked: 3/5 (0.0410s)
Cracked: 4/5 (0.0564s)
Cracked: 5/5 (0.0348s)
Calcul parallèle terminé
✓ Finished: mean=0.032100s ± 0.018240s

Résultats expérimentaux



Comparaison des sécurités

Groupe	$\mathbb{Z}/p\mathbb{Z}$	$\mathbb{G}$
Meilleur algorithme	Crible algébrique (NFS)	rho de Pollard
Complexité	$O\Big(\exp((\log p)^{1/3}(\log\log p)^{2/3})\Big)$	$O\big(\exp(\frac{n}{2})\big)$

Avantage des courbes elliptiques

Nfs n'est pas applicable aux courbes elliptiques car cet algorithme repose sur de la factorisation, ce qui n'existe pas avec les groupes construits sur une courbe elliptique.

Pour des clés de 256 bits:

- 2.575e+64 années-cœurs pour la force brute
- 1.656e+24 années-cœurs pour rho de Pollard
- 3177 années-cœurs pour NFS

Les courbes elliptiques permettent :

- Le cryptosystème d'ElGamal qui permet des communications sécurisés
- Des calculs plus performants pour une sécurité maximale par rapport à d'autres systèmes

Cependant, il est important de continuer de développer de nouveaux algorithmes car l'algorithme de Shor, qui se base sur les propriétés d'ordinateurs quantiques, permet de résoudre le problème du logarithme discret en temps polynomial, donc de casser la sécurité des courbes elliptiques.

Fin

Merci pour votre attention.

Annexes

```
class CourbeElliptique:
    def __init__(self, a, b, c, o):
        self.a = a
        self.b = b
        self.c = c
        self.o = o


    def __eq__(self, d):
        return self.a == d.a and self.b == d.b and self.c == d.c and self.o == d.o

class Point:
   def __init__(self, x, y, courbe_el):
       if not isinstance(courbe_el, CourbeElliptique):
           raise TypeError(f"courbe_el : a <CourbeElliptique> was expected, not a {type(courbe_el)}")

       self.CE = courbe_el

       self.x = x % courbe_el.o
       self.y = y % courbe_el.o


       if x != float("inf") and self not in courbe_el:
           raise ValueError(f"{courbe_el} doesn't contains a point with ({x}, {y}) mod {courbe_el.o} =
({self.x}, {self.y}) coordinates")
       self.courbe_el = courbe_el
```

Annexes

```
    def __add__(self, other):
        if self.courbe_el != other.courbe_el:
            raise ValueError(f"{self} and {other} aren't part of the same curb, they can't be add")

        return -(self*other)

    def __repr__(self):
        return f"({self.x}, {self.y}) from {self.courbe_el}"

    def __mul__(self, other):
        if isinstance(other, numbers.Integral):
            return self.mul_by_int(other)

        if isinstance(other, Point):
            return self.etoile(other)

        raise TypeError("a <Point> cannot be multiplied by something else than an <int> or a <Point>")

    def etoile(self, other):
        if self.courbe_el != other.courbe_el:
            raise ValueError(f"{self} and {other} aren't part of the same curb, * isn't defined for each
other")
```

Annexes

```
    if isinstance(other, Infini):
        return -self
    if self.x == other.x:
        if self.y != other.y or self.y == 0:
            return Infini(self.courbe_el)
        lamb = self.courbe_el.fp(self.x) * inv_mod(2*self.y, self.courbe_el.o)
    else:
        lamb = (self.y - other.y) * inv_mod(self.x - other.x, self.courbe_el.o)
    x3 = lamb ** 2 - self.courbe_el.a - self.x - other.x


    return Point(x3, lamb * x3 - lamb * self.x + self.y, self.courbe_el)


def mul_by_int(self, other):
    if other < 0:
        return -self * (-other)


    if other == 0:
        return Infini(self.courbe_el)


    sub = self * (other // 2)
    if other % 2:
        return sub + sub + self
    return sub + sub
```

Annexes

```
    def __rmul__(self, lamb):
        return self*lamb


    def __eq__(self, other):
        return (self.x, self.y, self.courbe_el) == (other.x, other.y, other.courbe_el)


    def __neg__(self):
        return Point(self.x, -self.y, self.courbe_el)


    def __sub__(self, other):
        return self + (-other)


    def ordre(self):
         inf = Infini(self.courbe_el)
         if self == inf:
             return 0
         k = 1
         while self*k != inf:
             k += 1
         return k

class Infini(Point):
    def __init__(self, courbe_el):
```

Annexes

```
        super().__init__(float("inf"), float("inf"), courbe_el)
        self.x = float("inf")
        self.y = float("inf")


    def etoile(self, other):
        return -other


    def __repr__(self):
        return f"Infini from {self.courbe_el}"


    def __neg__(self):
        return self

def find_points(c):
    l = []
    for x in range(c.o):
        for y in range(c.o):
            if y**2 % c.o == c.f(x) % c.o:
                print(f"Point trouvé : ({x},{y})")
                p = Point(x,y,c)
                l.append(p)
            if len(l) > 10000:
                return l
    return l
```

Annexes

```
def generate_PK(cle_secrete, P, CE):
    if isinstance(cle_secrete,numbers.Integral):
        B = cle_secrete * P  # entier * point
        return CE, P, B
    elif isinstance(cle_secrete,np.ndarray) and isinstance(P,np.ndarray):
        return np.array([generate_PK(cle_secrete[i],P[i],CE) for i in range(len(cle_secrete))])

def cryptage(cle_publique, message):
    CE, P, B = cle_publique
    k = randint(1, CE.o - 1)
    y1 = k*P
    y2 = message + k*B
    return y1, y2

def decryptage(message_encrypte, cle_secrete):
    y1, y2 = message_encrypte
    m = y2 - cle_secrete*y1
    return m

def crack_log_discret_force_brute(P,B,o):
    #On veut k tel que b^k = a mod p
    k = 1
    X = P
    while X != B:
        if k >= o:
            return None
        X = X+P
        k+=1
```

Annexes

```
    return k

def rho_de_pollard_CE(alpha, beta):
    if isinstance(alpha, Infini):
        raise ValueError("ordre(alpha) = 0 (point à l'infini)")

    n = ordre(alpha.CE)


    a = randint(0, n-1)
    b = randint(0, n-1)
    x = a*alpha + b*beta
    A = a
    B = b
    X = x

    def f(x, alpha, beta):
        r = x.x % 3
        if r == 0:
            return x + x
        elif r == 1:
            return x + alpha
        else:
            return x + beta

    def g(x, a, n):
        r = x.x % 3
        if r == 0:
```

Annexes

```
        return (2 * a) % n
    elif r == 1:
        return (a + 1) % n
    else:
        return a

def h(x, b, n):
    r = x.x % 3
    if r == 0:
        return (2 * b) % n
    elif r == 1:
        return b
    else:
        return (b + 1) % n

def appliquer(x, a, b):
    x2 = f(x, alpha, beta)
    a2 = g(x, a, n)
    b2 = h(x, b, n)
    return x2, a2, b2

while True:
    x, a, b = appliquer(x, a, b)

    X, A, B = appliquer(X, A, B)
    X, A, B = appliquer(X, A, B)

    if x == X:
```

Annexes

```
        r = (b - B) % n
        if r == 0:
            return "échec"
        inv_r = pow(r, -1, n)
        return (inv_r * ((A - a) % n)) % n

def kangourous(alpha: Point ,beta: Point ,a ,b ,p):

    N = int(np.sqrt(b-a)) + 1

    # saut
    def f(x:Point ):
        return (x.x + x.y) % N + 1

    # kangourous apprivoisés
    x = alpha*b
    dx = 0
    for i in range(N):
        fx = f(x)
        dx += fx
        x += alpha * fx


    # kangourous sauvages
    y = beta
    dy = 0
```

Annexes

```
        # les kangourous se rencontrent
        if y==x :
            return (b+dx-dy)%p

        # saut du kangourou sauvage
        fy = f(y)
        dy += fy
        y += alpha * fy

        # arrêt de la course entre les kangourous
        if dy>b-a+dx :
            print("échec -> changer l'interval ou f")
            return None

def extension(CE):
    return str(CE.a) + '_' + str(CE.b) + '_' + str(CE.c) + '_' + str(CE.o)

def nom_fichier_points(CE):
    return "pointsCEs/points_" + extension(CE) + ".txt"

def nom_fichier_ordre(CE):
    return "ordreptsCEs/ordrepts_" + extension(CE) + ".csv"

def nom_fichier_dico_direct(CE):
    return "dicos/dico_direct_"+ extension(CE) + ".txt"

def nom_fichier_dico_recip(CE):
    return "dicos/dico_recip_" + extension(CE) + ".txt"
```

Annexes

```
def nom_fichier_dico_ordre(CE):
    return "dicos_ordre/dico_ordre_" + extension(CE) + ".txt"

def nom_fichier_points_csv(CE):
    return "pointsCEs_csv/ordreptsliste_" + extension(CE) + ".csv"

def nom_fichier_point_ordre_max(CE):
    return "points_ordre_max/pt_om_" + extension(CE) + ".csv"


def nom_fichier_message_crypte(CE):
    return "messages_cryptes/message_crypte_" + extension(CE) + ".txt"

def text_to_pts(texte, dico):
    pts = []
    for e in texte:
        if e in dico:
            pts.append(dico[e])
    return pts

def pts_to_text(l, dico):
    texte = ''
    for e in l:
        if e in dico:
            texte += dico[e]
    return texte
```

Annexes

```python
def str_to_point(s, CE):
    s = s.strip()
    if not s.startswith("("):
        raise ValueError(f"Format invalide pour un point : {s}")
    coords = s.split(")")[0].replace("(", "").split(",")
    x, y = int(coords[0]), int(coords[1])
    return Point(x, y, CE)

def point_to_str(point):
    if not isinstance(point, Point):
        raise TypeError(f"Un objet de type Point est attendu, pas {type(point)}")
    return f"({point.x}, {point.y}) from {point.courbe_el}"

def points_to_list(CE):
    pts = []
    nom_f = nom_fichier_points(CE)
    try:
        with open(nom_f, "r", encoding="utf-8") as f:
            for ligne in f:
                ligne = ligne.strip()
                if not ligne:
                    continue
                try:
                    pt = str_to_point(ligne, CE)
                    pts.append(pt)
                except Exception:
                    # Ligne malformée ou non reconnue : on ignore
                    continue
```

Annexes

```
    except FileNotFoundError:
        print(f"⚠️ Fichier de points introuvable : {nom_f}")
    return pts


def cryptage_liste(message_pts, cle_publique):
    message_chiffre = []
    for pt in message_pts:
        message_chiffre.append(cryptage(cle_publique, pt))
    return message_chiffre

def decryptage_liste(message_chiffre, cle_secrete):
    message_dechiffre = []
    for couple in message_chiffre:
        message_dechiffre.append(decryptage(couple, cle_secrete))
    return message_dechiffre

def lire_dictionnaire(nom_fichier, CE=None):
    dico = {}
    with open(nom_fichier, "r", encoding="utf-8") as fichier:
        for ligne in fichier:
            if ":" not in ligne:
                continue
            cle, valeur = ligne.strip().split(":", 1)
            cle = cle.strip()
            valeur = valeur.strip()
```

Annexes

```
            # Si la valeur semble être un point "(x, y)"
            if valeur.startswith("(") and CE is not None:
                try:
                    valeur = str_to_point(valeur, CE)
                except Exception:
                    pass  # Si ce n'est pas un vrai point, on garde la chaîne
            dico[cle] = valeur
    print(f" Le dictionnaire a été chargé depuis '{nom_fichier}'")
    return dico

def sauvegarder_message_crypte(message_crypte,CE):
    nom_fichier = nom_fichier_message_crypte(CE)
    with open(nom_fichier, "w", encoding="utf-8") as fichier:
        for couple in message_crypte:
            y1, y2 = couple
            fichier.write(f"{point_to_str(y1)},{point_to_str(y2)}\n")
    print(f" Le message crypté a été sauvegardé dans '{nom_fichier}'")

def lire_message_crypte(nom_fichier, CE):
    message_crypte = []

    with open(nom_fichier, "r", encoding="utf-8") as fichier:
        for ligne in fichier:
            ligne = ligne.strip()
            if not ligne:
                continue

            # Trouver la position du second point "(" (celui de y2)
```

Annexes

```
            pos = ligne.find(") from Courbe elliptique")
            if pos == -1:
                raise ValueError(f"Ligne invalide (pas de point elliptique détecté) : {ligne}")

            # On cherche le début du deuxième point après le premier
            pos2 = ligne.find("(", pos + 1)
            if pos2 == -1:
                raise ValueError(f"Ligne invalide (2e point manquant) : {ligne}")

            y1_str = ligne[:pos + len(") from Courbe elliptique  y² = x³ + " + str(CE.a) + "x² + " + str(CE.c)
+ "mod " + str(CE.o))]
            y2_str = ligne[pos2:]

            try:
                y1 = str_to_point(y1_str, CE)
                y2 = str_to_point(y2_str, CE)
                message_crypte.append((y1, y2))
            except Exception as e:
                print(f" Erreur lors du décodage d'une ligne : {e}\n{ligne}")

    print(f" Le message crypté a été chargé depuis '{nom_fichier}'")
    return message_crypte

def envoyeur(message,cle_publique,CE,sauvarder=True):
    message = message.replace(" ", ";")
    nom_fichier = nom_fichier_message_crypte(CE)
    dico = lire_dictionnaire(nom_fichier_dico_direct(CE), CE)
    m2 = cryptage_liste(text_to_pts(message,dico), cle_publique)
```

Annexes

```
    if sauvarder:
        sauvegarder_message_crypte(m2,CE)
    return m2

def receveur(CE,cle_secrete, message_crypte=None):
    nom_dico_recip = nom_fichier_dico_recip(CE)
    dico_recip = lire_dictionnaire(nom_dico_recip, CEstand)
    if message_crypte is None:
        message_recu = nom_fichier_message_crypte(CE)
        message_crypte = lire_message_crypte(message_recu, CEstand)
    else :
        message_recu = message_crypte
    m4 = pts_to_text([str(p) for p in decryptage_liste(message_crypte,cle_secrete)], dico_recip)
    return m4.replace(";", " ")

def random_point(CE):
    nom_fichier = nom_fichier_points(CE)
    l = points_to_list(CE)
    if not l:
        raise ValueError(f"Aucun point trouvé dans '{nom_fichier}'")
    i = random.randint(0, len(l) - 1)
    return l[i]
```